Reproducing ConvMixer on CIFAR-10: Main Benchmark and Planned Ablations

Author Name

March 2026

Abstract

We reproduce the ConvMixer architecture of Trockman and Kolter [4] on CIFAR-10 and compare it against two compact reference baselines, ResNet18 [1] and DeiT-Tiny [3]. The implementation uses a shared PyTorch training pipeline with `timm`-provided baselines, Weights & Biases logging, and a `uv`-managed environment. In the current tracked runs, ConvMixer is the strongest model by test accuracy, ResNet18 is second, and DeiT-Tiny is clearly weaker under the same overall reproduction setting. At the same time, ConvMixer is slower per epoch than the other compared models in the present runs. The report also reserves space for follow-up ablations targeting the ConvMixer patch stem, channel mixing block, and data-size robustness; those experiments are already represented in code but their results will be added after the remaining runs are complete.

1 Introduction

ConvMixer was proposed as a deliberately simple image model that combines patch-based input processing with convolutional mixing operations [4]. The architecture is interesting because it removes much of the design complexity often associated with both modern convolutional networks and vision transformers, while still preserving separate spatial and channel mixing stages. This makes it a convenient target for a course-scale reproduction: the core model is compact, the implementation is short, and the architectural hypotheses are easy to probe with controlled modifications.

The goal of this project is twofold. First, we reproduce the main ConvMixer idea on CIFAR-10 and compare it against two lightweight but meaningful baselines: ResNet18 as a canonical residual CNN and DeiT-Tiny as a compact transformer-style vision model. Second, we prepare a set of focused ablations that isolate the role of the patch embedding stem and the channel mixing block. At the current stage, the main experiment is complete and logged in Weights & Biases, while the ablation section is scaffolded and will be populated as the remaining runs finish.

This report is intentionally framed as a constrained reproduction rather than a claim of exact paper-level parity. Our training budget, dataset, and implementation choices differ from the original large-scale setting, so the emphasis is on comparative behavior inside one common experimental pipeline rather than on matching the original headline numbers exactly.

2 Paper Background

ConvMixer is built around three ideas [4]. First, the input image is converted into a patch-level representation by a strided convolutional stem, which plays the role of patch embedding. Second, each block applies *depthwise* convolution to mix information spatially inside each channel. Third, a *pointwise* 1×1 convolution performs channel mixing. Nonlinearity and batch normalization are inserted between these operations, and the spatial mixing path is wrapped in a residual connection.

This structure is particularly convenient for ablation because the main functions of the architecture are already factorized. The patch stem can be replaced without changing the rest of the network, and the channel mixing stage can be swapped for either a stronger non-linear mixer or a weaker identity mapping while preserving the surrounding block layout.

The baseline choices serve different purposes. ResNet18 [1] represents a standard convolutional network with residual learning and remains a strong small-scale image baseline. DeiT-Tiny [3] provides a compact transformer-family comparison and tests whether a patch-oriented but non-convolutional alternative is competitive on this small dataset. Together, these models help position ConvMixer between classic CNN inductive bias and transformer-style patch processing.

3 Experimental Setup

Dataset. All current experiments use CIFAR-10 [2], a 10-class natural image benchmark with 32×32 inputs. This dataset is substantially smaller and lower-resolution than the settings emphasized in the original ConvMixer paper, so the present study should be read as a compact reproduction under limited compute rather than a direct scale-matched replication.

Models. The target architecture is ConvMixer with the default project configuration implemented in the repository: hidden dimension 256, depth 8, kernel size 5, and patch size 2. Two additional baselines are trained through the same script interface: ResNet18 and DeiT-Tiny. The latter two are instantiated via `timm` [5], while

ConvMixer is implemented directly in the repository in a small custom module.

Training pipeline. The project uses a shared trainer written in PyTorch. Runs are launched through a single command-line entry point, with optimizer, scheduler, logging, and checkpointing unified across models. The current trainer uses AdamW, cosine annealing over the configured epoch horizon, a batch size of 128 by default, and a fixed random seed of 42. Weights & Biases is used for metric logging, model comparison, and run tracking. Project dependencies are managed with `uv`.

Reproduction philosophy. The implementation deliberately favors one transparent training path over model-specific tuning. This keeps the comparison easy to inspect and helps separate architecture-level behavior from engineering variance. The downside is that a shared trainer is not guaranteed to be equally favorable to all three models, especially when a transformer-style model such as DeiT-Tiny is evaluated on a small low-resolution benchmark without the large-scale recipe for which it was originally designed.

Implementation notes. The current codebase also includes planned ablation variants for ConvMixer: a convolutional replacement for the patch stem, a non-linear channel mixer replacing the original pointwise convolution, an identity-based no-channel-mixing variant, and a reduced-data protocol. These ablations are already exposed through the training script, but only the main experiment is reported quantitatively in this draft.

Hardware and runtime context. The tracked runs were executed in a lightweight development environment and logged through Weights & Biases. Some intermediate debugging runs were performed on CPU in Colab-like settings during development. Because the currently compared runs may not be perfectly matched in epoch budget or runtime environment, the main comparison should be treated as informative rather than as a fully controlled benchmark.

4 Main Experiment Results

Table 1 summarizes the current three-model comparison. Exact values for best accuracy, completed epochs, and average epoch times should be copied from the finalized Weights & Biases run summaries before submission. They are intentionally left as explicit placeholders in this draft to avoid estimating quantitative values from screenshots.

The qualitative ordering is already clear from the current logged runs. ConvMixer achieves the highest test accuracy in the present setup and reaches its plateau relatively quickly. ResNet18 remains competitive and substantially stronger than DeiT-Tiny, but it does not reach the same top accuracy as ConvMixer. DeiT-Tiny improves steadily, yet under the current training configuration it lags well behind the other two models on CIFAR-10.

Runtime behavior tells a different story. The same Weights & Biases workspace shows that ConvMixer is slower per training epoch than both ResNet18 and DeiT-Tiny in the current runs, even though it is much smaller than ResNet18 in parameter count. This suggests that the depthwise-plus-pointwise structure is not automatically the cheapest option in wall-clock terms under the present implementation and execution environment.

The result pattern is also consistent with the intended interpretation of the benchmark. ResNet18 benefits from a strong convolutional prior and remains a reliable baseline on small natural-image datasets. DeiT-Tiny, in contrast, is more sensitive to recipe details and may need a more specialized optimization setup to close the gap. ConvMixer occupies an interesting middle ground: it keeps explicit convolutional mixing while adopting patch-based processing, and in this reproduction that combination appears to be the most effective among the tested options.

This comparison should still be interpreted with care. The current runs are useful for establishing a ranking inside one codebase, but they are not necessarily a perfectly matched benchmark. If the final ConvMixer, ResNet18, and DeiT-Tiny runs differ in epoch budget, scheduler horizon, or runtime backend, then the comparison remains valid as an empirical observation inside the reproduction pipeline but not as a fully controlled claim about architecture quality in general.

5 Ablation Study

The ablation code path is already prepared in the repository, but the full set of runs is still in progress. This section therefore records the intended experimental questions and leaves explicit space for the corresponding quantitative results.

These ablations are not arbitrary extensions. Each one targets a specific component that is easy to isolate in the ConvMixer design: the patch-forming front-end, the expressiveness of channel mixing, and the dependence of the model on training-set size. Together, they are meant to answer whether ConvMixer works well because of the exact combination proposed in the paper or because a broader family of simple patch-and-convolution hybrids is already strong on CIFAR-10.

5.1 Patch Embedding vs. Convolutional Stem

This ablation replaces the original patch embedding stem with a more conventional convolutional stem that preserves the downstream tensor shape. The goal is to test whether the patch projection itself is an important ingredient of ConvMixer, or whether a standard convolutional front-end can recover most of the same behavior once the later mixing blocks are unchanged.

Table 1: Main CIFAR-10 comparison for the current reproduction pipeline. Parameter counts are exact for the present model definitions; the remaining metrics should be copied from the corresponding finalized Weights & Biases runs.

Model	Params	Epochs	Best acc.	Train s/epoch	Eval s/epoch	Notes
ConvMixer	594 186	`TBD`	`TBD`	`TBD`	`TBD`	best acc., slowest
ResNet18	11 181 642	`TBD`	`TBD`	`TBD`	`TBD`	strong CNN baseline
DeiT-Tiny	5 489 482	`TBD`	`TBD`	`TBD`	`TBD`	weakest current run

Export the Weights & Biases plot for `best_test_accuracy` vs. epoch and save it as `report/figures/main_accuracy.pdf` or `report/figures/main_accuracy.png`.

The fallback screenshot should include only the three final runs: ConvMixer, ResNet18, and DeiT-Tiny.

Figure 1: Main training comparison from Weights & Biases. The preferred export is the `best_test_accuracy` plot for the three final runs.

Our working hypothesis is that the original patch-style stem is likely part of what makes ConvMixer distinctive, but the performance gap may be modest if the replacement stem remains shape-compatible and parameter-comparable. Results will be added after runs complete.

5.2 Pointwise Channel Mixing vs. Non-Linear Channel Mixing

The base ConvMixer block uses a single 1×1 convolution for channel mixing. The non-linear channel mixing ablation replaces that stage with a deeper non-linear mixer while keeping the rest of the block structure fixed. This tests whether the original pointwise mixer is sufficient, or whether additional channel-wise capacity materially changes performance.

The expected outcome is that this variant may improve flexibility but does not necessarily improve accuracy enough to justify its added complexity on CIFAR-10. Results will be added after runs complete.

5.3 Pointwise Channel Mixing vs. No Channel Mixing

This ablation removes the channel mixing stage entirely by replacing it with an identity mapping. It is the cleanest way to test whether the pointwise mixing path is essential, because the spatial depthwise branch and the classifier head remain intact.

The expected outcome is a noticeable drop in accuracy, which would support the interpretation that ConvMixer requires both spatial and channel mixing rather than spatial processing alone. Results will be added after runs complete.

5.4 Data-Size Robustness

The final planned ablation evaluates the same training pipeline under reduced fractions of the CIFAR-10 training set. This is intended to test whether ConvMixer retains its advantage when the amount of supervision decreases.

The current hypothesis is that smaller-data settings will amplify differences in inductive bias and optimization stability, potentially favoring the convolution-heavy models over DeiT-Tiny. Results will be added after runs complete.

6 Conclusion

This draft establishes the main reproduction result: in the current CIFAR-10 pipeline, ConvMixer is the strongest of the three evaluated models, ResNet18 is a solid second baseline, and DeiT-Tiny is clearly weaker under the present setup. The current logs also indicate that ConvMixer is slower per epoch than both comparison models, which is an important practical trade-off. The remaining ablation experiments are designed to determine which specific ConvMixer design choices — patch embedding, channel mixing, and data efficiency — are responsible for this behavior.

Optional runtime figure. If space permits, export the Weights & Biases comparison for `time/train_sec` and `time/eval_sec` and save it as `report/figures/main_runtime.pdf` or `report/figures/main_runtime.png`.

Figure 2: Optional runtime comparison from Weights & Biases. This figure is useful if the final draft still has room after the main accuracy plot and result table.

References

[1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[2] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[3] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *Proceedings of the International Conference on Machine Learning*, pages 10347–10357, 2021.

[4] Asher Trockman and J. Zico Kolter. Patches are all you need? *arXiv preprint arXiv:2201.09792*, 2022.

[5] Ross Wightman. Pytorch image models. https://github.com/rwightman/pytorch-image-models, 2019.